

EUROPEAN UNIVERSITY OF LEFKE

FACULTY OF ENGINEERING



COMP465

Advanced Computer Graphics

2D Ball Physics Simulation Project Report

Alper Enes YIRTIK(22124021)

Instructor: ASST. PROF. DR. Cem Burak Kalyoncu

TABLE OF CONTENTS

Advanced Computer Graphics.....	1
Introduction.....	2
General Program Structure.....	2
Vector Calculations.....	3
Ball Representation.....	3
Simulation Settings.....	3
Main Simulation Loop.....	4
Collision Handling.....	4
Rendering.....	5
Output and Results.....	5
Conclusion.....	5

Introduction

For this project, I developed a simple two dimensional physics simulation for the Advanced Computer Graphics course. The main reason I choose this project was to better understand how physics and graphics actually works together in a low level. Instead of using ready made libraries or engines, I wanted to see what happens when everything is implemented manually.

The simulation shows several balls moving inside an invisible rectangular area. Gravity pulls them downward and they collide both with each other and with the walls. The motion is not pre animated, it is fully calculated step by step. Each moment of the simulation is rendered as an image which makes it easy to observe how the system evolves over time.

General Program Structure

All program is written in a single C++ file. At first it might sound messy but in practice it made the project easier to manage. The code is splitted into logical parts. And each part focuses on one specific job like updating physics, checking collisions, or drawing objects on the screen.

This structure helped me to understand what's happening in each stage of the simulation and made debugging easier during development.

Vector Calculations

All movements and collision calculations are based on a two dimensional vector structure. This structure stores x and y values and supports the common operations like addition, subtraction, scaling, and normalization.

Using vectors instead of working with raw x and y values everywhere made the code cleaner. It also helped to keep the physics formulas close to their mathematical definitions which reduced mistakes and made the logic easier to understand.

Ball Representation

Every ball in the simulation has its own position, velocity, size, mass, and a color. The sizes are chosen randomly within a certain range, and the mass is calculated from the size of the ball. Bigger balls are heavier during collisions, which makes the behavior look more realistic.

The initial positions and velocities are also random. Because of this, every run of the program looks slightly different, which makes the simulation more interesting to watch.

Simulation Settings

The simulation runs inside a fixed screen area. There is gravity equal to all balls, pulling them downwards over time. When balls collide with each other, all of the collisions are elastic, meaning they bounce without losing energy. But wall collisions include some energy loss. So the motion slowly settles instead of continuing forever.

To avoid unstable behavior, physics calculations are divided into several smaller steps within each frame. It was helpful when balls were moving fast or when many collisions happened at the same time.

Main Simulation Loop

Every frame follows the same basic process. First, gravity is applied to update the velocities. Then, the positions of the balls are updated using their current velocities. After, collisions between balls are checked and resolved. At the end, collisions with the walls and ground are handled.

This entire process is repeated multiple times inside a single frame using smaller time steps. Doing this made the motion smoother and prevented balls from passing through each other.

Collision Handling

Ball to ball collisions are detected by checking how far the centers of two balls are from each other. If they are closer than they should be, a collision is happening. The first step is to separate the balls so they no longer overlap. This is important because overlapping objects may break the simulation.

When the balls are separated, their velocities are updated using an impulse based method. This approach takes the mass and relative velocity into account, so heavier balls have a stronger effect during collisions. This difference is clearly visible when balls with different sizes interact with each other.

Wall collisions are simpler. When a ball hits the wall, its position is corrected and velocity is reflected. A small energy loss is applied to the motion so it feels more natural.

Rendering

Rendering is done by using a basic filled circle drawing algorithm. For each ball, the program checks the pixels around its center and colors those that fall inside the radius. While this is not the most efficient method, it clearly shows how raster based drawing works and gives full control over the final image.

Also this part in this project helped me to understand how shapes are actually drawn at the pixel level.

Output and Results

Each frame of the simulation is saved as a PNG format. When these images are viewed in order, they became a smooth animation. The final result shows stable motion, collisions, and clear differences between balls with different sizes.

Also, even with many collisions happening at once, the simulation remains stable and visually consistent.

Conclusion

Overall, this project helped me to understand the connection between physics calculations and visual output in computer graphics. Implementing everything manually made the behavior of the system much clearer compared to using a ready made engine. The final simulation behaves realistically, remains stable, and produces a clean visual result.

Here is the source code of my project:

```

1  ✓ #include "Image.h"
2  #include <vector>
3  #include <cmath>
4  #include <string>
5  #include <algorithm>
6  #include <random>
7  #include <iostream>
8  #include <iomanip>
9  #include <sstream>
10
11
12  ✓ struct Vec2 {
13      double x, y;
14
15      Vec2() : x(0.0), y(0.0) {}
16      Vec2(double x, double y) : x(x), y(y) {}
17
18  ✓   Vec2 operator+(const Vec2& other) const {
19       |   return Vec2(x + other.x, y + other.y);
20       |   }
21
22  ✓   Vec2 operator-(const Vec2& other) const {
23       |   return Vec2(x - other.x, y - other.y);
24       |   }
25
26  ✓   Vec2 operator*(double scalar) const {
27       |   return Vec2(x * scalar, y * scalar);
28       |   }
29
30  ✓   Vec2 operator/(double scalar) const {
31       |   return Vec2(x / scalar, y / scalar);
32       |   }
33
34  ✓   double dot(const Vec2& other) const {
35       |   return x * other.x + y * other.y;
36       |   }
37
38  ✓   double length() const {
39       |   return std::sqrt(x * x + y * y);
40       |   }
41
42  ✓   double lengthSquared() const {
43       |   return x * x + y * y;
44       |   }
45
46  ✓   Vec2 normalized() const {
47       |   double len = length();
48       |   if (len > 0.0001) {

```

```

12 struct Vec2 {
46     Vec2 normalized() const {
48         if (len > 0.0001) {
49             return Vec2(x / len, y / len);
50         }
51         return Vec2(0, 0);
52     }
53 };
54
55
56 struct Ball {
57     Vec2 position;
58     Vec2 velocity;
59     double radius;
60     double mass;
61     RGBA color;
62
63     Ball() : position(0, 0), velocity(0, 0), radius(10.0), mass(1.0) {}
64 };
65
66 //sabitler
67 const int WIDTH = 800;
68 const int HEIGHT = 600;
69 const double GRAVITY = 500.0;
70 const double WALL_RESTITUTION = 0.9;
71 const double BALL_RESTITUTION = 1.0;
72 const int NUM_BALLS = 15;
73 const int TOTAL_FRAMES = 300;
74 const int SUB_STEPS = 4;
75 const double DT = 1.0 / 60.0; //saniyede 60 frame
76 const double SUB_DT = DT / SUB_STEPS;
77
78 //çemberi çizme algoritması
79 void drawFilledCircle(ColorImage& img, int cx, int cy, int radius, const RGBA& color) {
80     int minX = std::max(0, cx - radius);
81     int maxX = std::min(img.GetWidth() - 1, cx + radius);
82     int minY = std::max(0, cy - radius);
83     int maxY = std::min(img.GetHeight() - 1, cy + radius);
84
85     int radiusSq = radius * radius;
86
87     for (int y = minY; y <= maxY; y++) {
88         for (int x = minX; x <= maxX; x++) {
89             int dx = x - cx;
90             int dy = y - cy;
91             int distSq = dx * dx + dy * dy;
92
93             if (distSq <= radiusSq) {

```

```

91         int distSq = dx * dx + dy * dy;
92
93         if (distSq <= radiusSq) {
94             img(x, y) = color;
95         }
96     }
97 }
98 }
99
100 // top collisionları
101 void resolveBallCollision(Ball& ball1, Ball& ball2) {
102     Vec2 delta = ball1.position - ball2.position;
103     double dist = delta.length();
104     double minDist = ball1.radius + ball2.radius;
105
106     if (dist < minDist && dist > 0.0001) {
107
108         Vec2 normal = delta.normalized();
109         double overlap = minDist - dist;
110         Vec2 separation = normal * (overlap * 0.5);
111
112         ball1.position = ball1.position + separation;
113         ball2.position = ball2.position - separation;
114
115         Vec2 relativeVel = ball1.velocity - ball2.velocity;
116         double velAlongNormal = relativeVel.dot(normal);
117
118
119         if (velAlongNormal > 0) return;
120
121
122         double totalMass = ball1.mass + ball2.mass;
123         double impulseScalar = -(1.0 + BALL_RESTITUTION) * velAlongNormal;
124         impulseScalar /= totalMass;
125
126         Vec2 impulse = normal * impulseScalar;
127         ball1.velocity = ball1.velocity + impulse * ball2.mass;
128         ball2.velocity = ball2.velocity - impulse * ball1.mass;
129     }
130 }
131
132 //duvar collisionları
133 void handleWallCollision(Ball& ball) {
134     //sol duvar
135     if (ball.position.x - ball.radius < 0) {

```



```

133 void handleWallCollision(Ball& ball) {
134     //sol duvar
135     if (ball.position.x - ball.radius < 0) {
136         ball.position.x = ball.radius;
137         ball.velocity.x = -ball.velocity.x * WALL_RESTITUTION;
138     }
139
140     //sağ duvar
141     if (ball.position.x + ball.radius > WIDTH) {
142         ball.position.x = WIDTH - ball.radius;
143         ball.velocity.x = -ball.velocity.x * WALL_RESTITUTION;
144     }
145
146     //yukarı
147     if (ball.position.y - ball.radius < 0) {
148         ball.position.y = ball.radius;
149         ball.velocity.y = -ball.velocity.y * WALL_RESTITUTION;
150     }
151
152     //aşağı
153     if (ball.position.y + ball.radius > HEIGHT) {
154         ball.position.y = HEIGHT - ball.radius;
155         ball.velocity.y = -ball.velocity.y * WALL_RESTITUTION;
156     }
157 }
158
159
160 void updatePhysics(std::vector<Ball>& balls) {
161     //yerçekimi
162     for (auto& ball : balls) {
163         ball.velocity.y += GRAVITY * SUB_DT;
164     }
165
166     //pozisyon update
167     for (auto& ball : balls) {
168         ball.position = ball.position + ball.velocity * SUB_DT;
169     }
170
171     for (size_t i = 0; i < balls.size(); i++) {
172         for (size_t j = i + 1; j < balls.size(); j++) {
173             resolveBallCollision(balls[i], balls[j]);
174         }
175     }
176
177     for (auto& ball : balls) {
178         handleWallCollision(ball);
179     }
180 }

```

```

179     }
180 }
181
182 //rasgele renk
183 RGBA randomColor(std::mt19937& rng) {
184     std::uniform_int_distribution<int> colorDist(50, 255);
185     return RGBA(
186         colorDist(rng),
187         colorDist(rng),
188         colorDist(rng),
189         255
190     );
191 }
192
193 int main() {
194     std::random_device rd;
195     std::mt19937 rng(rd());
196     std::uniform_real_distribution<double> radiusDist(10.0, 25.0);
197     std::uniform_real_distribution<double> posXDist(50.0, WIDTH - 50.0);
198     std::uniform_real_distribution<double> posYDist(50.0, HEIGHT - 50.0);
199     std::uniform_real_distribution<double> velDist(-200.0, 200.0);
200
201     //top oluşturma
202     std::vector<Ball> balls(NUM_BALLS);
203     for (int i = 0; i < NUM_BALLS; i++) {
204         balls[i].radius = radiusDist(rng);
205         balls[i].mass = balls[i].radius * balls[i].radius;
206         balls[i].position = Vec2(posXDist(rng), posYDist(rng));
207         balls[i].velocity = Vec2(velDist(rng), velDist(rng));
208         balls[i].color = randomColor(rng);
209
210         //topların başlangıçta çakışmaması
211         for (int j = 0; j < i; j++) {
212             Vec2 delta = balls[i].position - balls[j].position;
213             double dist = delta.length();
214             double minDist = balls[i].radius + balls[j].radius;
215
216             if (dist < minDist) {
217
218                 Vec2 normal = delta.normalized();
219                 if (normal.length() < 0.0001) {
220                     normal = Vec2(1, 0);
221                 }
222                 balls[i].position = balls[j].position + normal * minDist;
223             }
224         }
225     }

```

```

225     }
226
227     //fotograf olusturma
228     ColorImage img(WIDTH, HEIGHT);
229
230
231     for (int frame = 0; frame < TOTAL_FRAMES; frame++) {
232
233         img.Clear();
234
235         //physics güncellemesi
236         for (int step = 0; step < SUB_STEPS; step++) {
237             updatePhysics(balls);
238         }
239
240         //topları çizme
241         for (const auto& ball : balls) {
242             int cx = static_cast<int>(ball.position.x);
243             int cy = static_cast<int>(ball.position.y);
244             int radius = static_cast<int>(ball.radius);
245             drawFilledCircle(img, cx, cy, radius, ball.color);
246         }
247
248         //frame'i kaydetme
249         std::ostringstream filename;
250         filename << "frame_" << std::setfill('0') << std::setw(3) << frame << ".png";
251         img.Save(filename.str());
252
253         //progress printleme
254         std::cout << "Frame " << (frame + 1) << "/" << TOTAL_FRAMES << " rendered" << std::endl;
255     }
256
257     std::cout << "Simulation completed and all frames saved." << std::endl;
258
259     return 0;
260 }
261
262

```